network configuration there. The Spring ecosystem provides everything we need to address those concerns, and it's probably what we would do in many scenarios. However, since we want to run our system on Kubernetes, we can manage those infrastructural concerns at the platform level and keep our applications simpler and more maintainable. That's where the Ingress API comes in handy.

An *Ingress* is an object that "manages external access to the services in a cluster, typically HTTP. Ingress may provide load balancing, SSL termination and name-based virtual hosting" (https://kubernetes.io/docs). An Ingress object acts as an entry point into a Kubernetes cluster and is capable of routing traffic from a single external IP address to multiple services running inside the cluster. We can use an Ingress object to perform load balancing, accept external traffic directed to a specific URL, and manage the TLS termination to expose the application services via HTTPS.

Ingress objects don't accomplish anything by themselves. We use an Ingress object to declare the *desired state* in terms of routing and TLS termination. The actual component that enforces those rules and routes traffic from outside the cluster to the applications inside is the *ingress controller*. Since multiple implementations are available, there's no default ingress controller included in the core Kubernetes distribution—it's up to you to install one. Ingress controllers are applications that are usually built using reverse proxies like NGINX, HAProxy, or Envoy. Some examples are Ambassador Emissary, Contour, and Ingress NGINX.

In production, the cloud platform or dedicated tools would be used to configure an ingress controller. In our local environment, we'll need some additional configuration to make the routing work. For the Polar Bookshop example, we'll use Ingress NGINX (https://github.com/kubernetes/ingress-nginx) in both environments.